

Precious Metals Portfolio — Value-at-Risk

Samuele Caneschi - MSc Finance & Energy Markets

Quantitative Risk Analysis

Sample period: 2019-01-01 to 2024-12-31

Abstract

This document is an in-depth companion to the Jupyter notebook `precious_metals_var.ipynb`. It documents the market-risk analysis of a \$1,000,000 equal-weight portfolio of Gold, Silver and Platinum COMEX futures: the data pipeline, the mathematics behind each risk metric, the exact code that implements it, and the economic interpretation a trader should attach to the output. The notebook computes Value-at-Risk by three independent methods (Historical Simulation, Parametric variance–covariance, and Monte Carlo), Expected Shortfall, a Kupiec backtest, four historical stress scenarios, and a rolling-window VaR, exporting every table to an Excel workbook and every chart to a 150-dpi PNG.

Contents

1	Portfolio definition and notation	2
2	Data pipeline (Section 0)	2
2.1	Loading: futures with ETF fallback	2
2.2	Descriptive statistics	2
3	Correlation and distribution (Section 1)	3
3.1	Static correlation	3
3.2	Rolling 90-day correlation	3
3.3	Normality testing	3
4	Value-at-Risk: three methods (Section 2)	3
4.1	Method 1 — Historical Simulation	4
4.2	Method 2 — Parametric (variance–covariance)	4
4.3	Method 3 — Monte Carlo simulation	5
4.4	Representative results	5
5	Expected Shortfall (Section 3)	5
6	Backtesting: Kupiec POF test (Section 4)	5
7	Stress scenarios (Section 5)	6
8	Rolling VaR (Section 6)	7
9	Outputs and reproducibility	8

1 Portfolio definition and notation

We hold three assets indexed $i \in \{\text{Gold, Silver, Platinum}\}$ with portfolio value $V = \$1,000,000$ and equal weights $w = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})^\top$.

Daily *log returns* are

$$r_i(t) = \ln\left(\frac{P_i(t)}{P_i(t-1)}\right), \quad (1)$$

preferred over simple returns because they are additive across time (the basis of the $\sqrt{h}$ horizon scaling) and feed directly into the normal-distribution machinery. The portfolio return and profit-and-loss are

$$r_p(t) = w^\top r(t) = \sum_i w_i r_i(t), \quad \text{P\&L}_p(t) = V \cdot r_p(t). \quad (2)$$

In code this is the entire bridge from returns to dollars:

```
1 portfolio_returns = returns @ WEIGHTS # weighted daily return
2 portfolio_pnl = portfolio_returns * PORTFOLIO_VALUE
```

2 Data pipeline (Section 0)

2.1 Loading: futures with ETF fallback

Each metal is downloaded from its COMEX futures ticker (GC=F, SI=F, PL=F); if a series fails, the loader falls back to the liquid ETF proxy (GLD, SLV, PPLT). There is *no synthetic fallback* — the analysis runs on real market data only, and the loader raises a clear error if both sources fail. Every series prints a source tag (LIVE or ETF).

```
1 def _try_download(ticker):
2     df = yf.download(ticker, start=START, end=END,
3                     progress=False, auto_adjust=True)
4     if isinstance(df.columns, pd.MultiIndex): # newer yfinance layout
5         close = df["Close"]
6         s = close.iloc[:, 0] if close.ndim > 1 else close
7     else:
8         s = df["Close"]
9     return s.dropna().astype(float)
10
11 for name in ASSETS:
12     s = _try_download(FUTURES[name]); tag = "live"
13     if s is None:
14         s = _try_download(ETFs[name]); tag = "ETF"
15     if s is None:
16         raise RuntimeError(f"Could not download {name} ...")
```

The `auto_adjust=True` flag returns split/dividend-adjusted closes, and the `MultiIndex` guard handles the column layout returned by recent `yfinance` versions. Prices are then aligned on common trading days (`dropna(how="any")`) so all three legs share an identical date index before returns are computed.

2.2 Descriptive statistics

For each asset the notebook reports N , mean and standard deviation of daily returns, annualised volatility $\sigma_{\text{ann}} = \sigma_{\text{daily}}\sqrt{252}$, skewness, and *raw* kurtosis (a normal distribution has kurtosis 3). A value well above 3 signals fat tails — the first hint that a Gaussian VaR will be too optimistic. Figure 1 shows the price and return panels.

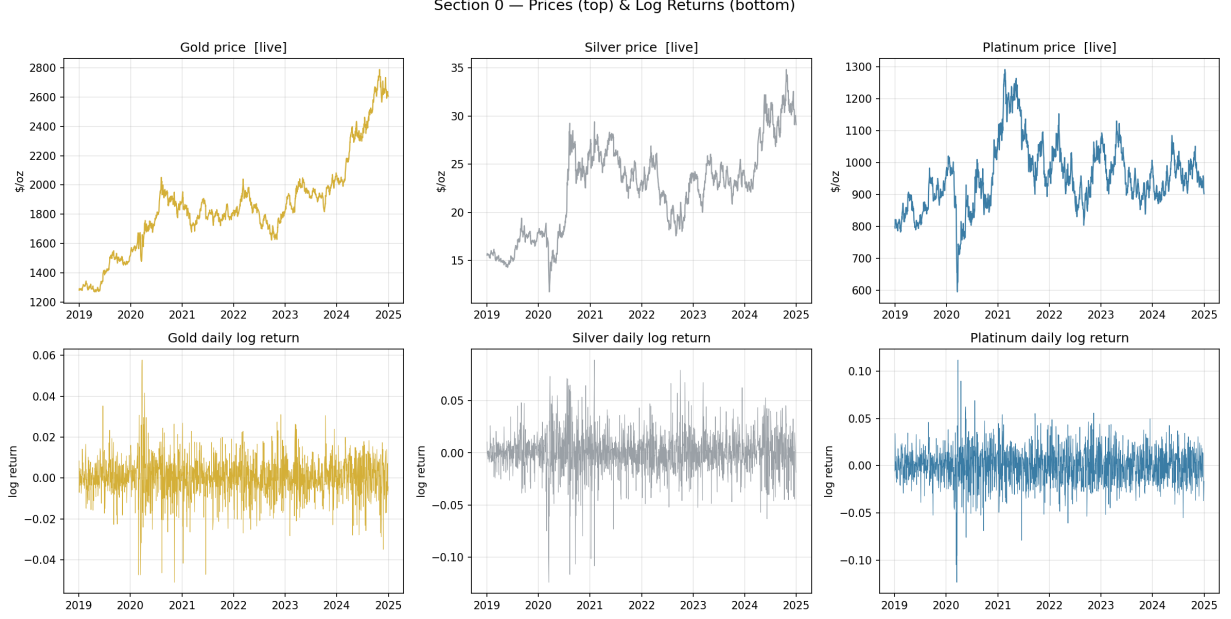


Figure 1: Section 0 — price levels (top) and daily log returns (bottom) for the three metals. Volatility clustering is visible around the 2020 COVID shock.

3 Correlation and distribution (Section 1)

3.1 Static correlation

The full-sample Pearson correlation matrix is computed on daily log returns, together with a matrix of p -values testing $H_0 : \rho = 0$. All off-diagonal correlations are positive — the three legs share a common precious-metals beta.

3.2 Rolling 90-day correlation

The key risk-management insight lives in the rolling window (Figure 2):

```

1 roll["Gold-Silver"] = returns["Gold"].rolling(90).corr(returns["Silver"])
2 roll["Gold-Platinum"] = returns["Gold"].rolling(90).corr(returns["Platinum"])
3 roll["Silver-Platinum"] = returns["Silver"].rolling(90).corr(returns["Platinum"])

```

Correlations spike towards 1 during the COVID crash and the Russia invasion: **diversification evaporates exactly when losses cluster**, so any VaR that assumes a stable correlation matrix will understate crisis risk.

3.3 Normality testing

Each return series is plotted against a fitted normal and submitted to the Jarque–Bera test, whose statistic combines skewness S and kurtosis K :

$$JB = \frac{n}{6} \left(S^2 + \frac{(K - 3)^2}{4} \right) \sim \chi_2^2 \quad \text{under } H_0 \text{ (normality)}. \quad (3)$$

For all three metals H_0 is decisively rejected ($p \ll 0.05$), confirming fat tails (Figure 3).

4 Value-at-Risk: three methods (Section 2)

VaR at confidence α over horizon h is the loss not exceeded with probability α . We report $\alpha \in \{95\%, 99\%\}$ at $h \in \{1, 10\}$ days, scaling one-day figures by the square-root-of-time rule

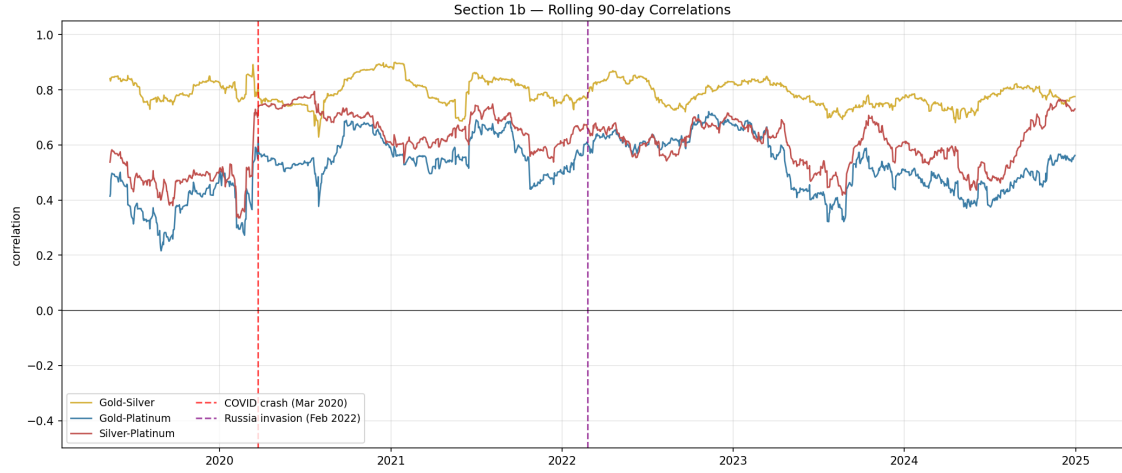


Figure 2: Section 1b — 90-day rolling correlations with crisis dates marked.

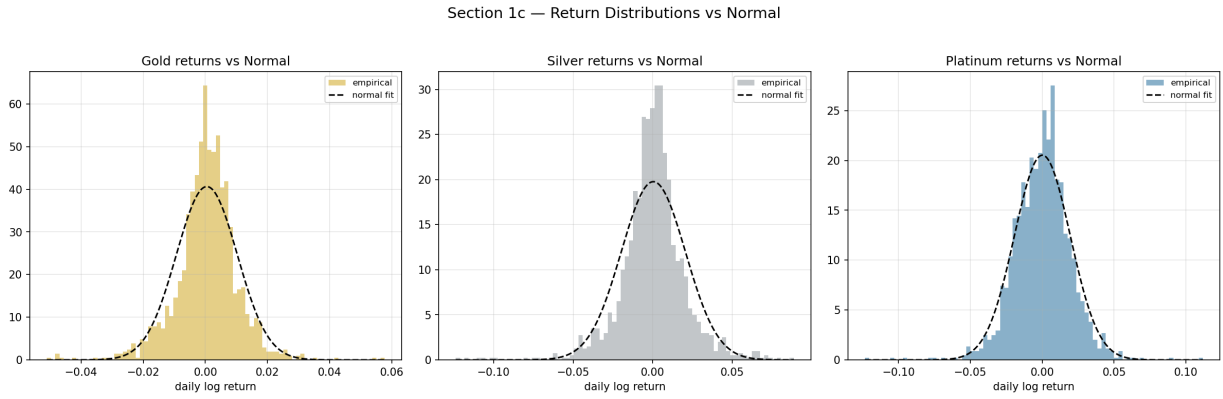


Figure 3: Section 1c — empirical return histograms vs. fitted normal density. The empirical peaks are taller and the tails heavier than Gaussian.

$$\text{VaR}_h = \text{VaR}_1 \sqrt{h} \text{ (here } \sqrt{10}\text{)}.$$

4.1 Method 1 — Historical Simulation

Non-parametric: the empirical percentile of the realised P&L distribution, making no distributional assumption and preserving the true fat tails and crisis days.

$$\text{VaR}_\alpha^{\text{HS}} = -\text{Percentile}(\text{P\&L}, (1 - \alpha) \cdot 100). \quad (4)$$

```
1 def var_historical(ret_series, conf, value):
2     pnl = ret_series.values * value
3     return -np.percentile(pnl, (1 - conf) * 100) # 95%→5th, 99%→1st
```

4.2 Method 2 — Parametric (variance-covariance)

Assumes returns are normal with mean μ and standard deviation σ :

$$\text{VaR}_\alpha^{\text{param}} = (-\mu + z_\alpha \sigma) V, \quad z_{95} = 1.645, z_{99} = 2.326. \quad (5)$$

```
1 def var_parametric(ret_series, conf, value):
2     mu, sig = ret_series.mean(), ret_series.std()
3     return (-mu + Z[conf] * sig) * value
```

Because it ignores kurtosis, this method *understates* VaR for fat-tailed assets; comparing it against Historical quantifies the “fat-tail tax”.

4.3 Method 3 — Monte Carlo simulation

Simulates 10,000 correlated return paths from the empirical mean vector μ and covariance matrix Σ . Correlation is imposed through the Cholesky factor L where $\Sigma = LL^\top$: if $Z \sim N(0, I)$ then $\mu + LZ$ has covariance Σ .

```
1 def var_montecarlo(mu_vec, cov_mat, w, conf, value, n_sims=10_000):
2     L = np.linalg.cholesky(cov_mat)
3     Zmat = np.random.standard_normal((n_sims, len(w)))
4     sim_r = mu_vec + Zmat @ L.T
5     sim_pnl = (sim_r @ w) * value
6     return -np.percentile(sim_pnl, (1 - conf) * 100), sim_pnl
```

The seed is fixed (`np.random.seed(42)`) for reproducibility. Each metal is also risked *standalone* at the full \$1 M notional so the figures are directly comparable; the gap between the sum of the standalone VaRs and the portfolio VaR is the **diversification benefit**.

4.4 Representative results

Table 1: Portfolio VaR by method (\$1 M equal-weight). Live data, 1,508 return observations. Parametric/Monte Carlo sit $\approx 12\%$ below Historical at 99%, the cost of ignoring fat tails.

Method	VaR 95% (1d)	VaR 99% (1d)	VaR 99% (10d)
Historical	\$22,962	\$37,985	\$120,121
Parametric	\$23,518	\$33,388	\$105,582
Monte Carlo	\$23,017	\$33,471	\$105,844

5 Expected Shortfall (Section 3)

Expected Shortfall (a.k.a. CVaR) is the *average* loss conditional on breaching VaR:

$$ES_\alpha = \mathbb{E}[L \mid L > \text{VaR}_\alpha]. \quad (6)$$

```
1 tail_hist = portfolio_pnl[portfolio_pnl < -var_hs_99]
2 ES_hist_99 = -tail_hist.mean()
3 es_ratio = ES_hist_99 / var_hs_99 # tail-risk amplifier
```

Unlike VaR, ES is *coherent* (sub-additive — it never penalises diversification), which is why Basel FRTB replaced VaR with ES at 97.5%. The **ES/VaR ratio** measures tail length: the realised value of 1.50 means that on the days VaR is breached the portfolio loses on average half again as much as the VaR line suggests (Figure 4).

6 Backtesting: Kupiec POF test (Section 4)

A correct 99% VaR is breached on $\approx 1\%$ of days. Let T be the number of observations and x the number of breaches, with $\hat{p} = x/T$ and target $p_0 = 0.01$. The Kupiec proportion-of-failures likelihood ratio is

$$\text{LR}_{\text{POF}} = -2 \ln \left[\frac{p_0^x (1 - p_0)^{T-x}}{\hat{p}^x (1 - \hat{p})^{T-x}} \right] \sim \chi_1^2 \quad \text{under } H_0 : \text{true rate} = p_0. \quad (7)$$

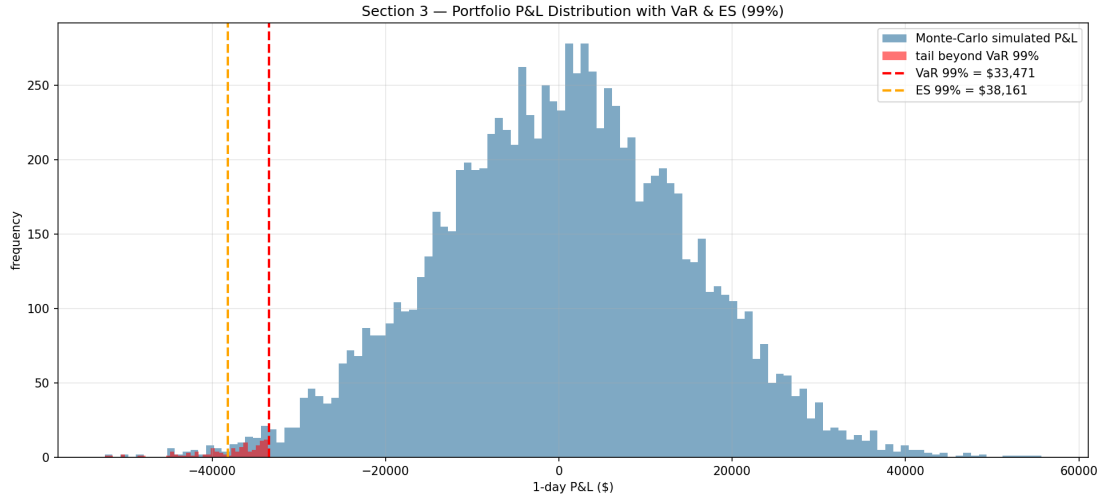


Figure 4: Section 3 — simulated P&L distribution; the red line is VaR 99%, the orange line ES 99%, and the shaded region the tail beyond VaR.

```

1 LR = -2 * (violations * np.log(p0 / p_hat) +
2           (T - violations) * np.log((1 - p0) / (1 - p_hat)))
3 p_value = 1 - chi2.cdf(LR, df=1)
4 conclusion = "ACCEPTED" if p_value > 0.05 else "REJECTED"

```

On live data the model produced 16 breaches (1.06% vs. 1.00% expected), $LR = 0.06$, $p = 0.81$ — comfortably **accepted**. The breaches cluster in the crisis windows (Figure 5), illustrating the known weakness of *unconditional* VaR that motivates conditional/EWMA extensions.

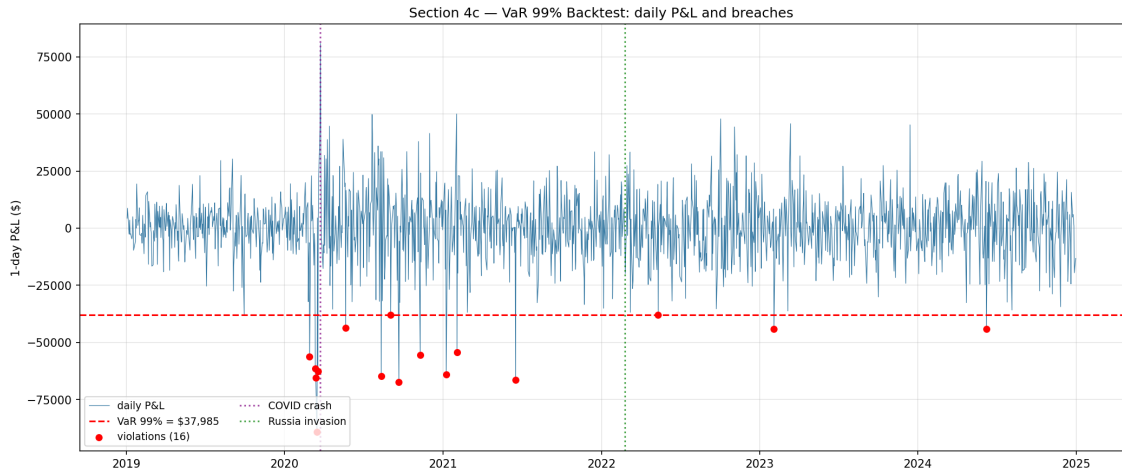


Figure 5: Section 4c — daily P&L with the VaR 99% threshold (dashed) and each breach marked (red dots).

7 Stress scenarios (Section 5)

Four real episodes are replayed through the current book. For each window the notebook reports total P&L, worst single day, and per-leg cumulative returns ($e^{\sum_t r_i(t)} - 1$).

The Russia case is instructive: gold's safe-haven bid nearly offset the loss, so the metals book behaved as a partial *hedge* rather than a source of loss. The silver squeeze shows idiosyncratic single-leg risk that a portfolio-level VaR can mask (Figure 6).

Table 2: Historical stress scenarios (\$1 M equal-weight).

Scenario	Days	Portfolio P&L	Gold	Silver	Platinum
COVID crash	23	−\$272,527	−2.5%	−27.7%	−37.4%
Russia invasion	14	−\$17,182	+1.0%	+2.3%	−8.1%
Gold flash crash	2	−\$54,903	−4.5%	−8.0%	−3.4%
Silver squeeze	5	+\$3,954	−1.1%	+3.3%	−1.0%

Section 5 — Stress Scenario P&L

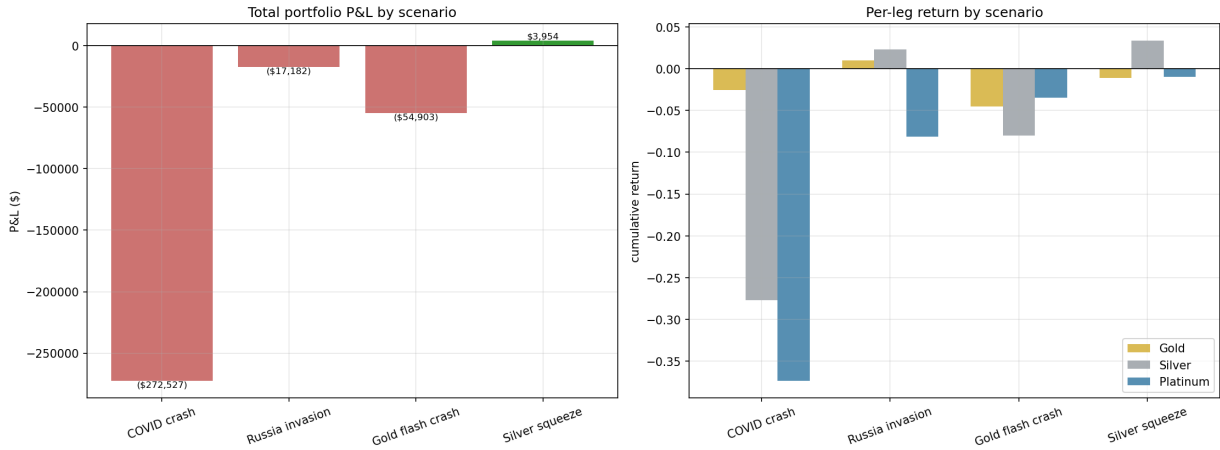


Figure 6: Section 5 — total portfolio P&L (left) and per-leg returns (right) by scenario.

8 Rolling VaR (Section 6)

A 252-day (one trading year) rolling Historical VaR 99% is recomputed each day:

```

1 for t in range(252, len(portfolio_pnl)):
2     window = pnl_vals[t-252:t]
3     roll_var.append(-np.percentile(window, 1))

```

This turns a static risk number into a live speedometer (Figure 7). Risk ramped to its peak ($\approx \$65,000$) in late 2020 as the COVID shock sat inside the window, stayed elevated for a year (window memory), then decayed to a $\approx \$23,500$ trough by 2024. It tells the desk when to resize the book.

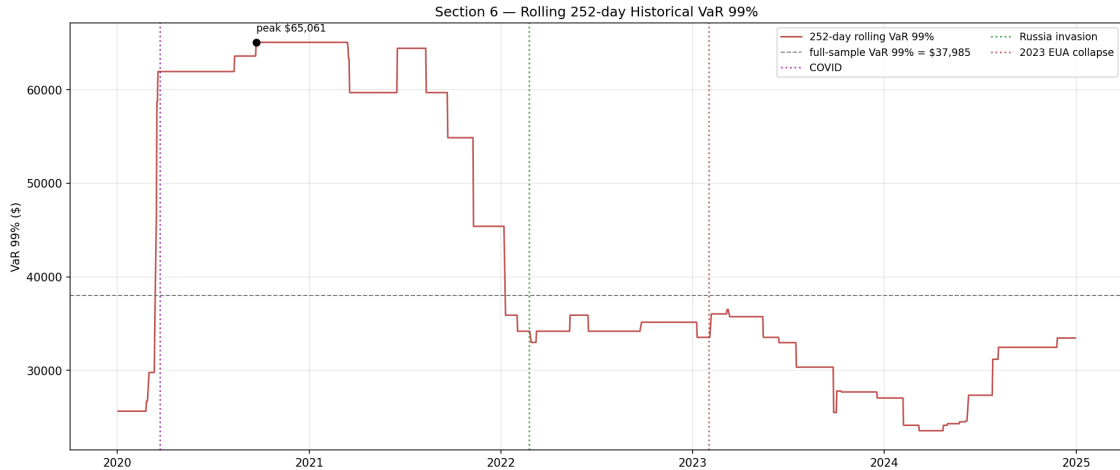


Figure 7: Section 6 — 252-day rolling Historical VaR 99% vs. the full-sample estimate.

9 Outputs and reproducibility

Excel workbook. A single `ExcelWriter` writes `precious_metals_var_results.xlsx` with seven sheets: `S0>Returns`, `S1`Correlations`, `S2`VaR_Summary`, `S3`ES`, `S4`Backtesting`, `S5`Stress`, `S6`RollingVaR`.

Figures. All seven charts are saved as 150-dpi PNGs (`fig0_prices_returns.png`...`fig6_rolling_var.png`).

Reproducibility. The Monte Carlo seed is fixed at 42; the notebook is generated programmatically by `build_pm_var_notebook.py` and executes end-to-end without manual intervention.

Headline takeaways.

- Portfolio VaR 99% (1d, Historical) $\approx$ \$37,985 (3.8% of NAV); 10-day $\approx$ \$120,121.
- Parametric VaR sits $\approx$ 12% *below* Historical — the fat-tail understatement, consistent with the rejected Jarque–Bera tests.
- ES 99% $\approx$ \$57,162, an ES/VaR ratio of 1.50.
- Kupiec test accepts the model (16 breaches, 1.06% vs. 1.0%).
- Worst stress: the COVID crash, $-\$272,527$ over 23 days — roughly seven times the 1-day 99% VaR, underlining why stress testing complements VaR.